4 Retrieval Models

4.1 Introduction

- Retrieval Models Provide a mathematical framework for defining the search process
  - Basis of many ranking algorithms
- Progress in retrieval models has corresponded with improvements in effectiveness Theories about relevance
- Relevance
  - Complex concept that has been studied for some time
    - Many factors to consider
    - People often disagree when making relevance judgments
  - Retrieval models make various assumptions about relevance to simplify problem
  - Examples
    - Topical vs. user relevance
      - Topical relevance
        - A document is topically relevant to a query if it is judged to be on the same topic
        - Example: A web page containing a biography of Abraham Lincoln
          - Topically relevant to the query “Abraham Lincoln”
          - Topically relevant to the queries “U.S. presidents” and “Civil War”
      - User relevance
        - Takes into account all the other factors that go into a user’s judgment of relevance.
          - Age of the document, the language of the document, intended target audience, the novelty of the document, etc.
        - Example: A document containing just a list of all the U.S. presidents
          - Topically relevant to the query “Abraham Lincoln”
          - May not be considered relevant to the person who submitted the query because they were looking for more detail on Lincoln’s life
    - Binary vs. multi-valued relevance
      - Binary relevance
        - A document is either relevant or not relevant

- Multi-value relevance
  - Some documents are less relevant than others, but still more relevant than documents that are completely off-topic
  - Example
    - A document containing a list of U.S. presidents could be less topically relevant than the Lincoln biography, but certainly more relevant than an advertisement for a Lincoln automobile
  - Multiple levels of relevance are certainly important in evaluation, when people are asked to judge relevance
    - Even just three levels (relevant, non-relevant, unsure) has been shown to make the judges' task much easier

4.2 Retrieval Model Overview

- Older models
  - Boolean retrieval
  - Vector Space model
- Probabilistic Models

4.3 Older Models

4.3.1 Boolean Retrieval

- Two possible outcomes for query processing
  - TRUE and FALSE
  - "Exact-match" retrieval
  - No ranking at all
- Query usually specified using Boolean operators
  - AND, OR, NOT
  - Proximity operators are also used
    - Proximity operator specifies that words must occur together within a specific distance (usually in terms of word count)
    - e.g., "tropical fish", or "find tropical within 5 words of fish"

- Advantages
  - Results are predictable, relatively easy to explain
  - Many different document features (other than words) can be incorporated in the query specification
    - e.g., metadata such as a document date or document type
  - Efficient processing since many documents can be eliminated from search
- Disadvantages
  - Effectiveness depends entirely on users
  - Simple queries usually don't work well (because of the lack of a sophisticated ranking algorithm)
    - All documents containing the specified query words will be retrieved, and this retrieved set will be presented to the user in some order, such as by publication date, that has little to do with relevance

Complex queries are difficult to construct

Examples of Boolean Query Formulation

Domain: search engine that has indexed a collection of news stories

Queries

- lincoln
  - Would retrieve a large number of documents that mention
    - Lincoln cars
    - Places named Lincoln
    - Stories about President Lincoln
    - etc.
  - All of these documents would be equivalent in terms of ranking in the Boolean retrieval model, regardless of how many times the word "lincoln" occurs or in what context it occurs
- president AND lincoln (scope of search narrowed)
  - Query retrieves a set of documents that contain both words, occurring anywhere in the document
    - Including for example, stories involving the management of the Ford Motor Company and Lincoln cars
      - e.g., the document: Ford Motor Company today announced that Darryl Hazel will succeed Brian Kelley as president of Lincoln Mercury

- president AND lincoln AND NOT (automobile OR car)
  - The user is trying to eliminate documents about cars by using the NOT operator
  - This would remove any document that contains even a single mention of the words “automobile” or “car” anywhere in the document
  - The use of the NOT operator, in general, removes too many relevant documents along with non-relevant documents and is not recommended
    - For example, important documents like documents containing the following will not be retrieved: **Lincoln’s body departs Washington in a nine-car funeral train**
- president AND lincoln AND biography AND life AND birthplace AND gettysburg AND NOT (automobile OR car)
  - Here, the user may be trying to further narrow the query by adding in additional words that should occur in biographies
    - Unfortunately, in a Boolean search engine, putting too many search terms into the query with the AND operator often results in nothing being retrieved
    - To avoid this, the user may try using an OR instead:
- president AND lincoln AND (biography OR life OR birthplace OR gettysburg) AND NOT (automobile OR car)
  - This will retrieve any document containing the words “president” and “lincoln”, along with any one of the words “biography”, “life”, “birthplace”, or “gettysburg” (and does not mention “automobile” or “car”).
- Even if this query is effective at retrieving relevant documents, we still can’t specify which words are more important or that having more of the associated words is better than any one of them

4.3.2 Vector Space Model

- Documents and query represented by a vector of term weights

$$D_i = (d_{i1}, d_{i2}, \ldots, d_{it}),$$

d_{ij} represents the weight of the jth term

- Collection represented by a matrix of term weights

$$\begin{array}{cccccc} & Term_1 & Term_2 & \ldots & Term_t \\ Doc_1 & d_{11} & d_{12} & \ldots & d_{1t} \\ Doc_2 & d_{21} & d_{22} & \ldots & d_{2t} \\ \vdots & \vdots & & & \\ Doc_n & d_{n1} & d_{n2} & \ldots & d_{nt} \end{array}$$

Example

D_1 Tropical Freshwater Aquarium Fish.
D_2 Tropical Fish, Aquarium Care, Tank Setup.
D_3 Keeping Tropical Fish and Goldfish in Aquariums, and Fish Bowls.
D_4 The Tropical Tank Homepage - Tropical Fish and Aquariums.

Terms	**Documents**			
	D_1	**D_2**	**D_3**	**D_4**
aquarium	1	1	1	1
bowl	0	0	1	0
care	0	1	0	0
fish	1	1	2	1
freshwater	1	0	0	0
goldfish	0	0	1	0
homepage	0	0	0	1
keep	0	0	1	0
setup	0	1	0	0
tank	0	1	0	1
tropical	1	1	1	2

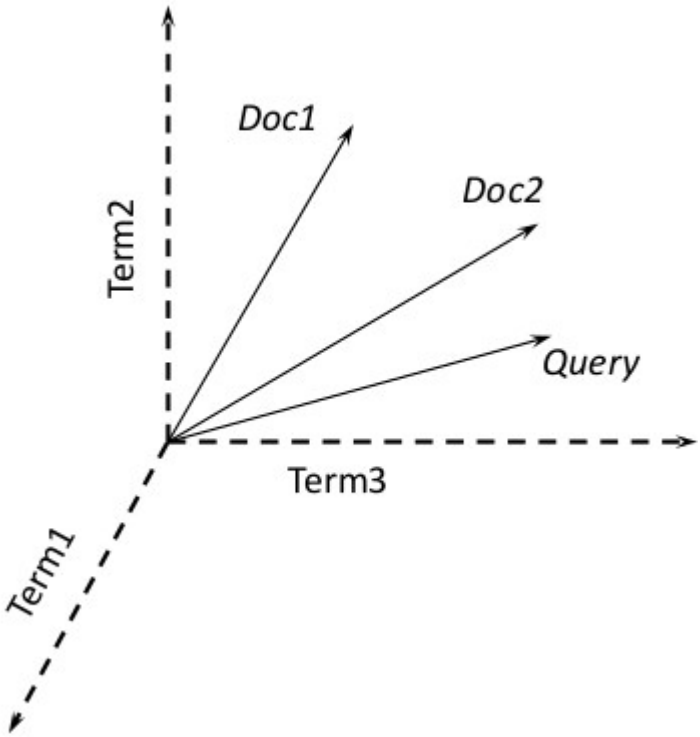
- Document D3 represented by the vector: $(1, 1, 0, 2, 0, 1, 0, 1, 0, 0, 1)$.
- Queries represented similarly to documents

$$Q = (q_1, q_2, \ldots, q_t),$$

where q_j is the weight of the jth term in the query.

- Query “tropical fish” represented as $(0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1)$.

- Simple diagrams used to to visualize the documents and queries (typically, as points or vectors in a three-dimensional picture)



- But this can be misleading for high- dimensional space
  - the t terms represent all the document features that are indexed
  - In enterprise and web applications, this corresponds to hundreds of thousands or even millions of dimensions

- Documents ranked by **distance** between points representing query and documents
- Similarity measure more common (rather) than a distance or dissimilarity measure
  - e.g. Cosine correlation

$$Cosine(D_i, Q) = \frac{\sum_{j=1}^{t} d_{ij} \cdot q_j}{\sqrt{\sum_{j=1}^{t} {d_{ij}}^2 \cdot \sum_{j=1}^{t} {q_j}^2}}$$

- The numerator
  - Dot product or inner product
  - The sum of the products of the term weights for the matching query and document terms
- The denominator
  - Normalizes the score by dividing by the product of the lengths of the two vectors
- There is no theoretical reason why the cosine correlation should be preferred to other similarity measures, but it does perform somewhat better in evaluations of search quality

Similarity Calculation Example

- Consider two documents D1 and D2, and a query Q
  - D1 = (0.5, 0.8, 0.3)
  - D2 = (0.9, 0.4, 0.2)
  - Q = (1.5, 1.0, 0)

$$
\begin{aligned}
Cosine(D_1, Q) &= \frac{(0.5 \times 1.5) + (0.8 \times 1.0)}{\sqrt{(0.5^2 + 0.8^2 + 0.3^2)(1.5^2 + 1.0^2)}} \\
&= \frac{1.55}{\sqrt{(0.98 \times 3.25)}} = 0.87
\end{aligned}
$$

$$
\begin{aligned}
Cosine(D_2, Q) &= \frac{(0.9 \times 1.5) + (0.4 \times 1.0)}{\sqrt{(0.9^2 + 0.4^2 + 0.2^2)(1.5^2 + 1.0^2)}} \\
&= \frac{1.75}{\sqrt{(1.01 \times 3.25)}} = 0.97
\end{aligned}
$$

  - Q more similar to D2 than D1

Term Weights

- Many different weighting schemes have been tried over the years
- Most of these are variations on tf.idf weighting

- The term frequency component, tf, reflects the importance of a term in a document Di (or query). This is usually computed as a normalized count of the term occurrences in a document, for example

$$tf_{ik} = \frac{f_{ik}}{\sum_{j=1}^{t} f_{ij}}$$

where tf_{ik} is the term frequency weight of term k in document D_i, and f_{ik} is the number of occurrences of term k in the document.

  - Notes
    - A document collection can contain documents of many different lengths
    - Normalization accounts for this to some degree
    - But long documents can have many terms occurring once and others occurring hundreds of times
    - Retrieval experiments have shown that to reduce the impact of these frequent terms, it is effective to use the **logarithm** of the number of term occurrences in tf weights rather than the raw count (see below)

- The inverse document frequency component (idf) reflects the importance of the term in the collection of documents
  - The more documents that a term occurs in, the less discriminating the term is between documents and, consequently, the less useful it will be in retrieval
  - The typical form of this weight is

$$idf_k = \log \frac{N}{n_k}$$

where idf_k is the inverse document frequency weight for term k, N is the number of documents in the collection, and n_k is the number of documents in which term k occurs.

- The two terms are then multiplied together

- Most common form of the tf.idf expression

$$d_{ik} = \frac{(\log(f_{ik}) + 1) \cdot \log(N/n_k)}{\sqrt{\sum_{k=1}^{t} [(\log(f_{ik}) + 1.0) \cdot \log(N/n_k)]^2}}$$

- The form of query term weighting is essentially the same as for document term weighting
- Notes
  - Adding 1 to the term frequency component ensures that terms with frequency 1 have a non-zero weight
  - In the model, term weights are computed only for terms that occur in the document (or query)
  - Although there is no explicit definition of relevance in the vector space model, there is an implicit assumption that relevance is related to the similarity of query and document vectors
    - i.e., documents "closer" (or more similar) to the query are more likely to be relevant
      - This is primarily a model of topical relevance, although features related to user relevance could be incorporated into the vector representation

Advantages of the model

- Simple computational framework for ranking
- Any similarity measure or term weighting scheme could be used

Disadvantages

- Assumption of term independence
- No predictions about techniques for effective ranking

4.3.3 Probabilistic Models

4.3.3.1 Introduction

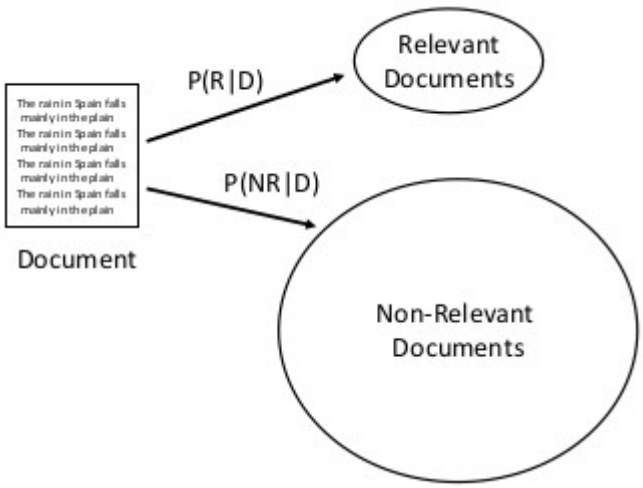
- In probabilistic approaches to IR, the occurrence of a query term in a document D contributes to the probability that D will be judged as relevant
- The weight assigned to a query term should be based on the expected value of that contribution
- **Probability Ranking Principle – Robertson (1977)**
  - Dominant paradigm of probabilistic models

- "If a reference retrieval system's response to each request is a ranking of the documents in the collection in order of decreasing probability of relevance to the user who submitted the request,
- where the probabilities are estimated as accurately as possible on the basis of whatever data have been made available to the system for this purpose,
- the overall effectiveness of the system to its user will be the best that is obtainable on the basis of those data."

  - The Probability Ranking Principle, however, does not address how to estimate the probability of relevance

4.3.3.2 IR as Classification

- Assuming relevance is binary, there will be two sets of documents for each query
  - Relevant documents and non-relevant documents
- Given a new document, the task of a search engine would be to decide whether the document belongs in the relevant set or the non-relevant set
  - i.e, classify the document as relevant or non-relevant, and retrieve it if it is relevant
    - i.e., classify the document into the set (relevant or not relevant) that has the highest probability
    - i.e., decide that a document D is relevant if P(R|D) > P (NR|D), where P(R|D) is a conditional probability representing the probability of relevance given the representation of that document, and P (NR|D) is the conditional probability of non-relevance



Bayes Decision Rule

- A document D is relevant if $P(R \mid D) > P(NR \mid D)$, where $P(R \mid D)$ and $P(NR \mid D)$ are conditional probabilities

Estimating probabilities

- Use Bayes Rule

based on the probability of occurrence of words in D that are in R

$$P(R|D) = \frac{P(D|R)P(R)}{P(D)}$$

A prior probability of relevance

- Classify a document as relevant if

$$\frac{P(D|R)}{P(D|NR)} > \frac{P(NR)}{P(R)}$$

Based on the Bayes Decision Rule $P(R \mid D) > P(NR \mid D)$

- L.H.S. is the likelihood ratio of D being *relevant*

Estimating $P(D \mid R)$

Assume *word independence* and use *individual term* (d_i) *probabilities*

$$P(D|R) = \prod_{i=1}^{t} P(d_i|R)$$

Binary (weights in doc) independence (of word) model

- Document represented by a vector of *binary* features indicating *term occurrence* (or non-occurrence)
- p_i is *probability* that term i occurs in relevant document, s_i is *probability of occurrence* in non-relevant document

Binary independence model

- document represented by a vector of binary features indicating term occurrence (or non-occurrence)
- p_i is probability that term i occurs (i.e., has value 1) in relevant document, s_i is probability of occurrence in non-relevant document

$$\frac{P(D|R)}{P(D|NR)} = \prod_{i:d_i=1} \frac{p_i}{s_i} \cdot \prod_{i:d_i=0} \frac{1-p_i}{1-s_i}$$

where $\prod_{i:d_i=1}$ means that it is a product over the terms that have the value 1 in the document. We can now do a bit of mathematical manipulation to get:

$$\prod_{i:d_i=1} \frac{p_i}{s_i} \cdot \left(\prod_{i:d_i=1} \frac{1-s_i}{1-p_i} \cdot \prod_{i:d_i=1} \frac{1-p_i}{1-s_i} \right) \cdot \prod_{i:d_i=0} \frac{1-p_i}{1-s_i}$$

$$= \prod_{i:d_i=1} \frac{p_i(1-s_i)}{s_i(1-p_i)} \cdot \prod_{i} \frac{1-p_i}{1-s_i}$$

Binary Independence Model

Computing the likelihood ratio of *D* using p_i and s_i

Probability of *i* in *R*

Probability of *i* in *NR*

Probability of *i not* in *R*

$$\frac{P(D|R)}{P(D|NR)} = \prod_{i:d_i=1} \frac{p_i}{s_i} \cdot \prod_{i:d_i=0} \frac{1-p_i}{1-s_i}$$

word *i* in *D*

word *i* not in *D*

Probability of *i not* in *NR*

$$= \prod_{i:d_i=1} \frac{p_i}{s_i} \cdot \left(\prod_{i:d_i=1} \frac{1-s_i}{1-p_i} \cdot \prod_{i:d_i=1} \frac{1-p_i}{1-s_i} \right) \cdot \prod_{i:d_i=0} \frac{1-p_i}{1-s_i}$$

$$= \boxed{\prod_{i:d_i=1} \frac{p_i(1-s_i)}{s_i(1-p_i)}} \cdot \cancel{\prod_{i} \frac{1-p_i}{1-s_i}}$$

(Same for all documents)

Binary independence model contd.

- Scoring function is $\sum_{i:d_i=1} \log \frac{p_i(1-s_i)}{s_i(1-p_i)}$
  - Using log to avoid multiplying lots of small numbers
- Query provides information about relevant documents
- ***If there is no other information on the relevant set***
  - pi is set to a constant (= 0.5)
  - si (= ni/N), where ni is the number of documents including term i in the non-relevant set
  - si is approximated by the entire collection (=N), similar to IDF-like weight

$$\log \frac{0.5(1-\frac{n_i}{N})}{\frac{n_i}{N}(1-0.5)} = \log \frac{N-n_i}{n_i}$$

- ***If we do have information about term occurrences*** in the relevant and non-relevant sets, it can be summarized in a contingency table

	Relevant	Non-relevant	Total
$d_i = 1$	r_i	$n_i - r_i$	n_i
$d_i = 0$	$R - r_i$	$N - n_i - R + r_i$	$N - n_i$
Total	R	$N - R$	N

- Given this table
  - Estimates for pi = ri /R (the number of relevant documents that contain a term divided by the total number of relevant documents)
  - Estimate for si = (ni − ri)/(N − R) (the number of non-relevant documents that contain a term divided by the total number of non-relevant documents)
- These estimates could cause a problem if some of the entries in the contingency table were zeros (e.g., ri = 0 results in a term weight of 0)
- To avoid this, a standard solution is to add 0.5 to each count and 1 to the totals
- Hence
  - pi = (ri + 0.5)/(R + 1)
  - si = (ni −ri +0.5)/(N −R+1.0)

Contingency Table

	Relevant	Non-relevant	Total
$d_i = 1$	r_i	$n_i - r_i$	n_i
$d_i = 0$	$R - r_i$	$N - n_i - R + r_i$	$N - n_i$
Total	R	$N - R$	N

where r_i = number of relevant documents containing term i
n_i = number of documents in a collection containing i
R = number of relevant documents in the collection
N = number of documents in the collection

$$p_i = (r_i + 0.5)/(R + 1) \qquad s_i = (n_i - r_i + 0.5)/(N - R + 1)$$

Gives *scoring* function:

$$\sum_{i:d_i=1} \log \frac{p_i(1-s_i)}{s_i(1-p_i)}$$

$$\approx \sum_{i:d_i=q_i=1} \log \frac{(r_i+0.5)/(R-r_i+0.5)}{(n_i-r_i+0.5)/(N-n_i-R+r_i+0.5)}$$

23

4.3.3.3 BM25 Ranking Algorithm

Popular and effective ranking algorithm based on binary independence model

See text for details

4.3.4 Ranking Based on Language Models

Language models are used to represent text in a variety of language technologies, such as speech recognition, machine translation, and handwriting recognition

- Unigram language model
  - probability distribution over the words in a language
  - generation of text consists of pulling words out of a “bucket” according to the probability distribution and replacing them
- N-gram language model

    - some applications use bigram and trigram language models where probabilities depend on previous words
- A topic in a document or query can be represented as a language model
    - i.e., words that tend to occur often when discussing a topic will have high probabilities in the corresponding language models
- Multinomial distribution over words
    - text is modeled as a finite sequence of words, where there are t possible words at each point in the sequence
    - commonly used, but not only possibility
    - doesn’t model burstiness
    - Burstiness
        - The observation that once a word is “pulled out of the bucket,” it tends to be pulled out repeatedly